

Mastering TensorFlow: Day 4 - Working with Datasets

using `tf.data`

Recap of Day 3

Yesterday, we explored tensor operations, slicing, and indexing. Today, we move forward to learn how to handle datasets in TensorFlow.

```
import tensorflow as tf
```

```
import numpy as np
```

```
# Creating a simple dataset with NumPy
```

```
data = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
labels = np.array([0, 1, 2])
```

```
# Displaying the data and labels
```

```
print("Data:\n", data)
```

```
print("Labels:", labels)
```

```
↗ Data:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Labels: [0 1 2]
```

```
import tensorflow as tf
```

```
# Converting the data to TensorFlow tensors
```

```
data_tensor = tf.constant(data, dtype=tf.float32)
```

```
labels_tensor = tf.constant(labels, dtype=tf.int32)
```

```
# Displaying the tensors
```

```
print("Data Tensor:\n", data_tensor)
```

```
print("Labels Tensor:", labels_tensor)
```

```
↗ Data Tensor:
tf.Tensor(
[[1. 2. 3.]
 [4. 5. 6.]
 [7. 8. 9.]], shape=(3, 3), dtype=float32)
```

```
Labels Tensor: tf.Tensor([0 1 2], shape=(3,), dtype=int32)
```

```
# Normalizing the data by dividing by the maximum value (in this case, 9)
normalized_data_tensor = data_tensor / 9.0

# Displaying the normalized data
print("Normalized Data Tensor:\n", normalized_data_tensor)
```

```
Normalized Data Tensor:
tf.Tensor(
[[0.11111111 0.22222222 0.33333334]
 [0.44444445 0.55555556 0.66666667]
 [0.77777778 0.88888889 1.          ]], shape=(3, 3), dtype=float32)
```

```
# Adding a constant value to the normalized data tensor
added_tensor = tf.add(normalized_data_tensor, 1.0)

# Displaying the result
print("Added Tensor:\n", added_tensor)
```

```
Added Tensor:
tf.Tensor(
[[1.11111112 1.22222222 1.33333334]
 [1.44444444 1.55555556 1.66666667]
 [1.77777778 1.88888888 2.          ]], shape=(3, 3), dtype=float32)
```

Explanation for Each Cell:-

Simple Dataset Creation: Introduces the idea of datasets using basic NumPy arrays, which are easy to understand and visualize.

Converting to Tensors: Demonstrates the conversion from NumPy arrays to TensorFlow tensors, a foundational step in working with TensorFlow.

Basic Preprocessing: Shows how to normalize data, an essential preprocessing step, in a very straightforward manner.

Simple Tensor Operations: Applies tensor operations to the dataset, helping reinforce concepts from Day 3 in a practical context.

```
# Loading a sample dataset from TensorFlow datasets
from tensorflow.keras.datasets import mnist

# Load the data
(x_train, y_train), (x_test, y_test) = mnist.load_data()
print("Training data shape:", x_train.shape)
print("Test data shape:", x_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/11490434/11490434 — 0s 0us/step  
Training data shape: (60000, 28, 28)  
Test data shape: (10000, 28, 28)
```

```
# Creating a TensorFlow dataset  
train_dataset = tf.data.Dataset.from_tensor_slices((x_train, y_train))  
  
# Viewing the first element in the dataset  
for image, label in train_dataset.take(1):  
    print("Image shape:", image.shape)  
    print("Label:", label)
```

```
Image shape: (28, 28)  
Label: tf.Tensor(5, shape=(), dtype=uint8)
```

```
# Normalizing the data  
def normalize(image, label):  
    image = tf.cast(image, tf.float32) / 255.0  
    return image, label  
  
# Applying the normalization  
train_dataset = train_dataset.map(normalize)  
  
# Batch the dataset  
train_dataset = train_dataset.batch(32)  
  
# Display the shape of a batch  
for image_batch, label_batch in train_dataset.take(1):  
    print("Image batch shape:", image_batch.shape)  
    print("Label batch shape:", label_batch.shape)
```

```
Image batch shape: (32, 28, 28)  
Label batch shape: (32,)
```

Conclusion

Today, we learned how to load and preprocess datasets using TensorFlow's `tf.data` API. This is a crucial step in preparing data for training machine learning models. Tomorrow, we'll explore how to build and train a simple neural network using this preprocessed data. Stay tuned! 🚀

